Bevy on iOS

with Mads Marquart / @madsmtm

Accessing iOS and macOS platform APIs in Rust using `objc2`.

Contents

1. Platform APIs?
2. Small introduction to Objective-C.
3. Various Objective-C patterns.
4. `'static` callbacks in Bevy.

Platform APIs?

`objc2` in Bevy

Component	Crate	Underlying platform crates
Windowing and Input	`winit`	`objc2-ui-kit` / `objc2-app-kit`
Rendering	`wgpu`	`objc2-metal`
Audio	`cpal`	`objc2-core-audio`
Gamepads	`gilrs`	`objc2-io-kit`

Could also be useful for…

- Notifications / Widgets / similar system GUI stuff
- Neural Engine / built-in ML
- App Store purchases
- Haptic feedback
- Bluetooth / Wi-Fi / NFC / etc.
- TouchID authentication
- Accelerometer / Gyroscope
- …

Small introduction to Objective-C

Example: Opening a URL in the web browser with `UIKit`.

What is Objective-C?

- Object-oriented superset of C.
- The language that a large amount of Apple’s system APIs are written in.

Objective-C declarations

Objetive-C declarations for this:

```
@interface UIApplication
+ (UIApplication*)sharedApplication;
- (void)openURL:(NSURL*)url
        options:(NSDictionary<NSString*, id> *)options
        completionHandler:(void (^)(BOOL success))completion;
@end
```

Objective-C's weird method call syntax

```
UIApplication* app = [UIApplication sharedApplication];

[app openURL:[NSURL URLWithString:@"https://example.com"]
     options:@{}
     completionHandler:^(BOOL success) {
         if (!success) {
             NSLog(@"Failed to open URL");
         }
     }];
```

How does that look with `objc2`?

```
let app = UIApplication::sharedApplication(mtm);

app.openURL_options_completionHandler(
    &NSURL::URLWithString(ns_string!("https://example.com")),
    &NSDictionary::new(),
    Some(&RcBlock::new(|success| {
        if !success {
            eprintln!("Failed to open URL");
        }
    })),
);
```

Demo

Various Objective-C patterns

Things that are nice to know when using these APIs.

`MainThreadMarker`

```
fn my_system(_: NonSendMarker) {
    let mtm = MainThreadMarker::new().unwrap();
    let app = UIApplication::sharedApplication(mtm);
}
```

Delegates

```
define_class!(
    #[unsafe(super(NSObject))]
    struct MyAppDelegate;

    unsafe impl UIApplicationDelegate for MyAppDelegate {
        #[unsafe(method(applicationWillTerminate:))]
        fn applicationWillTerminate(
            &self, application: &UIApplication,
        ) { /* .. */ }
    }
);
```

Memory management

`Retained` ≈ `Arc`, everything is refererence-counted.

```
struct Foo {
    app: Retained<UIApplication>,
}

fn get_foo(app: &UIApplication) -> Foo {
    Foo { app: app.retain() }
}

foo.app.doThing(...);
```

Completion handlers

```
fn doWithCompletionHandler(
    &self,
    completion: &Block<'static, fn(*mut NSError)>,
);

thing.doWithCompletionHandler(&RcBlock::new(|error| {
    if let Some(error) = unsafe { error.as_ref() } {
        eprintln!("failed xyz: {error}");
    }
    // Handle result.
}));
```

NSNotificationCenter

```
let center = NSNotificationCenter::defaultCenter();
let observer =
center.addObserverForName_object_queue_usingBlock(
    Some(UIContentSizeCategoryDidChangeNotification),
    None, // No sender filter.
    None, // No queue, run on posting thread's runloop.
    &RcBlock::new(|notification: NonNull<NSNotification>| {
        let notification = unsafe { notification.as_ref() };
        // ...
    }),
);
```

A wackier demo

Playing a game within UI notifications.

`'static` callbacks in Bevy

The problem

How do you go from:

```
pub fn open(
    url: &str,
    callback: impl FnOnce(bool) + 'static,
);
```

To something that feels natural in Bevy?

My “solution”

```
#[derive(Message)]
pub struct Opened {
    pub url: String,
    pub success: bool,
}
```

My “solution”

```
#[derive(Resource)]
pub struct Browser(mpsc::Sender<Opened>);
impl Browser {
    pub fn open(&self, url: &str) {
        let sender = self.0.clone();
        let url = url.to_string();
        open(&url, move |success| {
            sender.send(Opened { url, success }).unwrap();
        });
    }
}
```

My “solution”

```
// Inside `Plugin::build`:
let mut receiver = SyncCell::new(receiver);
app.add_systems(
    PreUpdate,
    move |mut writer: MessageWriter<Opened>| {
        for msg in receiver.get().try_iter() {
            writer.write(msg);
        }
    }
);
```

My “solution”

```rust
fn setup(browser: Res<Browser>) {
    browser.open("http://example.com");
}

fn handle_opened(mut reader: MessageReader<Opened>) {
    for Opened { url, success } in reader.read() {
        if !success {
            eprintln!("failed opening {url:?}");
        }
    }
}
```

An actually useful demo

Auto text size for accessibility.

Thanks for listening!

GitHub: https://github.com/madsmtm/objc2

Matrix: https://matrix.to/#/#objc2-users:matrix.org

The hidden slides

How would we call this from Rust?

```
void open_url(
    const char* url,
    void (*callback)(void*, bool),
    void* context,
) {
    UIApplication* app = [UIApplication sharedApplication];
    [app openURL: /* ... */
         completionHandler:^(BOOL success) {
             callback(context, !!success);
         }];
}
```

How would we call this from Rust?

```
// build.rs

fn main() {
    println!("cargo::rerun-if-changed=src/open_url.m");
    cc::Build::new()
        .file("src/open_url.m")
        .arg("-fobjc-arc")
        .compile("open_url");
}
```

How would we call this from Rust?

```
unsafe extern "C" {
    fn open_url(
        url: *const c_char,
        callback: extern "C" fn(*mut c_void, bool),
        context: *mut c_void,
    );
}
pub fn open(url: &str, callback: impl FnOnce() + 'static) {
    let callback = Box::new(callback);
    open_url(CString::new(url).unwrap().as_ptr(), /* ... */);
}
```

Problems?

- It’s cumbersome
- It’s inefficient
- It’s incomplete
- It’s untenable